LAB 7 – FUZZ TESTING FOR INPUT VALIDATION BUGS

Project: Electricity Bill Application
Name: Prashant Devendra Naik

SRN: PES1UG24AM199
Repository: L6_L7_17_electricity_bill

---

1. Objective

To understand and perform fuzz testing using the Hypothesis library in Python. The aim is to identify input validation errors, fix the bugs, and verify the application using Pytest.

2. Introduction

Fuzz testing is a software testing technique in which a program is tested with unexpected or invalid inputs to find errors.

In this lab, three functions of the electricity bill application were tested:

1. units_consumed() – Calculates electricity consumption.
2. daily_average() – Calculates average daily consumption.
3. tariff_rate() – Calculates the electricity tariff**.**

3. Implementation

Step 1: Install Dependencies

The required libraries were installed using:

python -m pip install hypothesis pytest

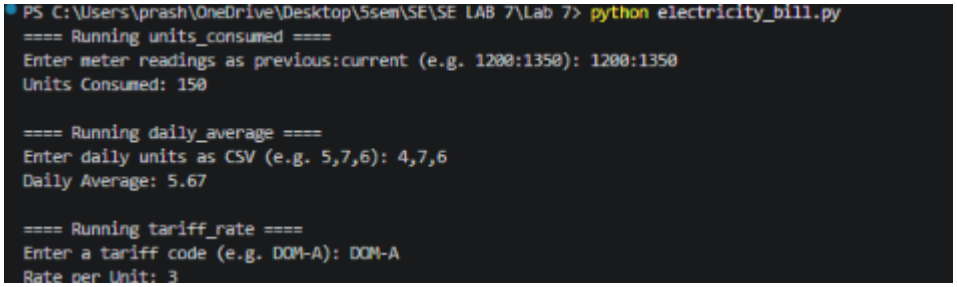
Step 2: Run the Original Application

The original application was executed using:

python electricity_bill.py

Valid inputs were provided, and the application produced the expected results.

Screenshot 1 – Original Application



Step 3: Create Fuzz Tests

A file named test_electricity_bill.py was created using Hypothesis.

Three test functions were written to check invalid meter readings, daily units, and tariff codes.

Step 4: Run Initial Tests

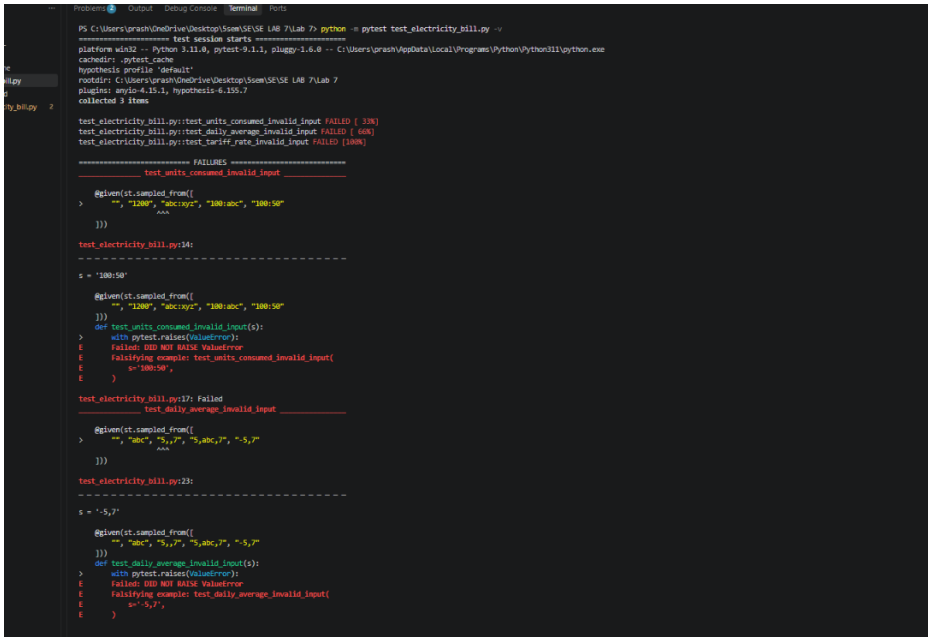
The tests were executed using:

python -m pytest test_electricity_bill.py -v

All three tests initially failed, revealing the following bugs:

- Negative electricity consumption was accepted.
- Negative daily units were accepted.
- An invalid tariff category caused a KeyError.

Screenshot 2 – Initial Failed Tests



```
Problems 2  Output  Debug Console  Terminal  Ports

PS C:\Users\prash\OneDrive\Desktop\Ssem\SE\SE LAB 7\Lab 7> python -m pytest test_electricity_bill.py -v
========== test session starts ==========
platform win32 -- Python 3.11.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\prash\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\prash\OneDrive\Desktop\Ssem\SE\SE LAB 7\Lab 7
plugins: anyio-4.15.1, hypothesis-6.155.7
collected 3 items

test_electricity_bill.py::test_units_consumed_invalid_input FAILED [ 33%]
test_electricity_bill.py::test_daily_average_invalid_input FAILED [ 66%]
test_electricity_bill.py::test_tariff_rate_invalid_input FAILED [100%]

========== FAILURES ==========
_____ test_units_consumed_invalid_input _____

    @given(st.sampled_from([
>       "", "1200", "abc:xyz", "100:abc", "100:50"
                    ^^^
    ]))

test_electricity_bill.py:14:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

s = '100:50'

    @given(st.sampled_from([
        "", "1200", "abc:xyz", "100:abc", "100:50"
    ]))
    def test_units_consumed_invalid_input(s):
>       with pytest.raises(ValueError):
E       Failed: DID NOT RAISE ValueError
E       Falsifying example: test_units_consumed_invalid_input(
E           s='100:50',
E       )

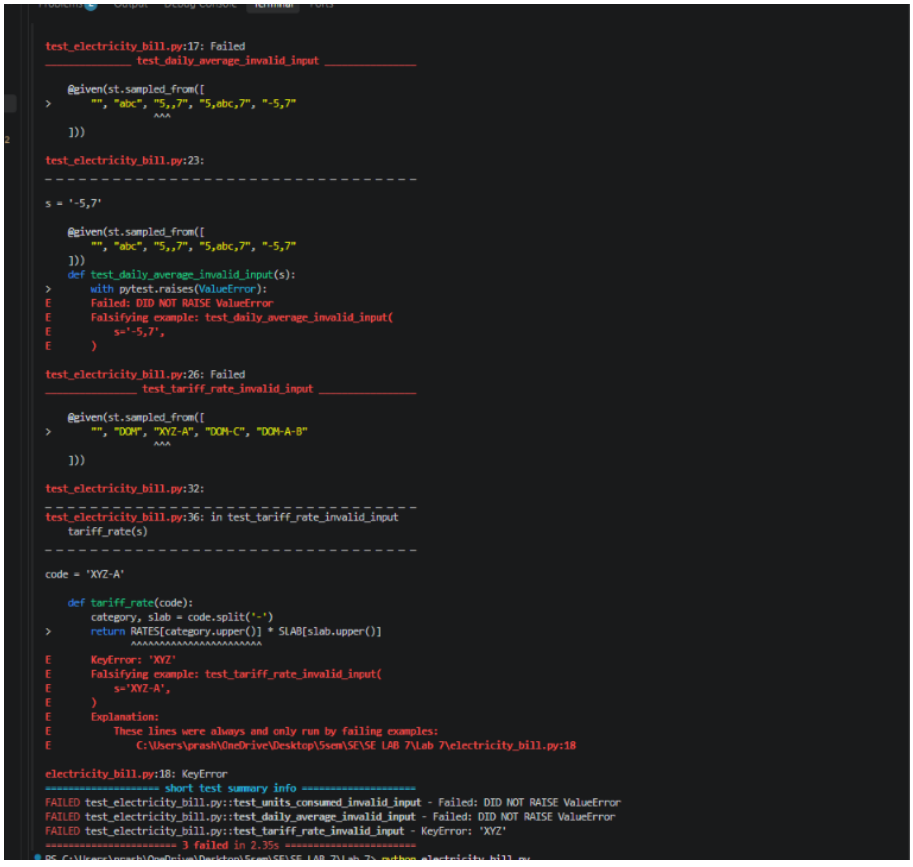
test_electricity_bill.py:17: Failed
_____ test_daily_average_invalid_input _____

    @given(st.sampled_from([
>       "", "abc", "5,,7", "5,abc,7", "-5,7"
                   ^^^
    ]))

test_electricity_bill.py:23:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

s = '-5,7'

    @given(st.sampled_from([
        "", "abc", "5,,7", "5,abc,7", "-5,7"
    ]))
    def test_daily_average_invalid_input(s):
>       with pytest.raises(ValueError):
E       Failed: DID NOT RAISE ValueError
E       Falsifying example: test_daily_average_invalid_input(
E           s='-5,7',
E       )
```



```
test_electricity_bill.py:17: Failed
_____ test_daily_average_invalid_input _____

    @given(st.sampled_from([
>       "", "abc", "5,,7", "5,abc,7", "-5,7"
                   ^^^
    ]))

test_electricity_bill.py:23:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

s = '-5,7'

    @given(st.sampled_from([
        "", "abc", "5,,7", "5,abc,7", "-5,7"
    ]))
    def test_daily_average_invalid_input(s):
>       with pytest.raises(ValueError):
E       Failed: DID NOT RAISE ValueError
E       Falsifying example: test_daily_average_invalid_input(
E           s='-5,7',
E       )

test_electricity_bill.py:26: Failed
_____ test_tariff_rate_invalid_input _____

    @given(st.sampled_from([
>       "", "DOM", "XYZ-A", "DOM-C", "DOM-A-B"
                   ^^^
    ]))

test_electricity_bill.py:32:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
test_electricity_bill.py:36: in test_tariff_rate_invalid_input
    tariff_rate(s)
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

code = 'XYZ-A'

    def tariff_rate(code):
        category, slab = code.split('-')
>       return RATES[category.upper()] * SLAB[slab.upper()]
               ^^^^^^^^^^^^^^^^^^^^^^^^

E       KeyError: 'XYZ'
E       Falsifying example: test_tariff_rate_invalid_input(
E           s='XYZ-A',
E       )
E       Explanation:
E           These lines were always and only run by failing examples:
E               C:\Users\prash\OneDrive\Desktop\Ssem\SE\SE LAB 7\Lab 7\electricity_bill.py:18

electricity_bill.py:18: KeyError
========== short test summary info ==========
FAILED test_electricity_bill.py::test_units_consumed_invalid_input - Failed: DID NOT RAISE ValueError
FAILED test_electricity_bill.py::test_daily_average_invalid_input - Failed: DID NOT RAISE ValueError
FAILED test_electricity_bill.py::test_tariff_rate_invalid_input - KeyError: 'XYZ'
========== 3 failed in 2.35s ==========
```

Step 5: Fix the Bugs

The electricity_bill.py file was updated with proper input validation.

The corrections included:

- Checking meter reading formats and values.
- Rejecting negative readings and consumption.
- Validating tariff categories and slabs.
- Displaying proper error messages for invalid inputs.

Step 6: Run the Fixed Application

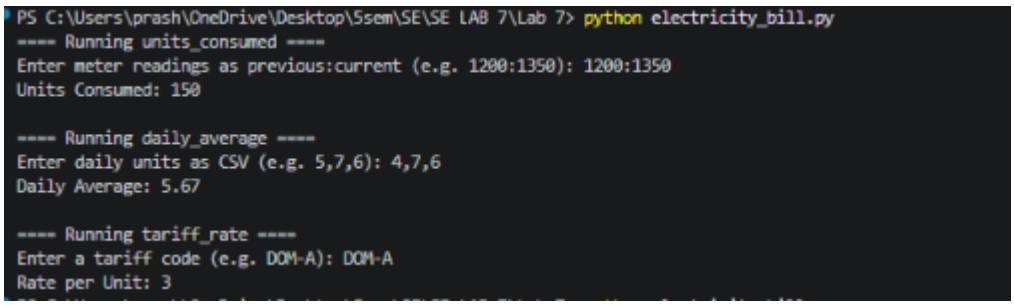
The corrected application was executed again:

python electricity_bill.py

It produced the expected results:

- Units consumed: 150
- Daily average: 5.67
- Tariff rate: 3

Screenshot 3 – Fixed Application



Step 7: Run Tests Again

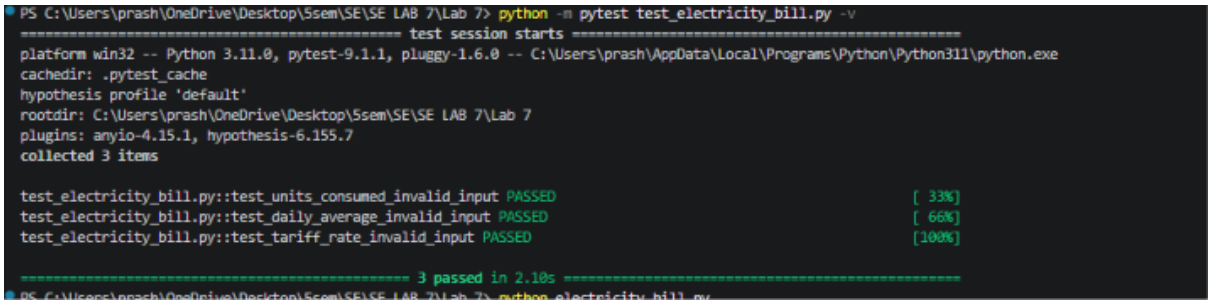
The test file was executed again:

python -m pytest test_electricity_bill.py -v

All three test functions passed successfully.

Final Result: 3 PASSED

Screenshot 4 – Successful Fuzz Tests



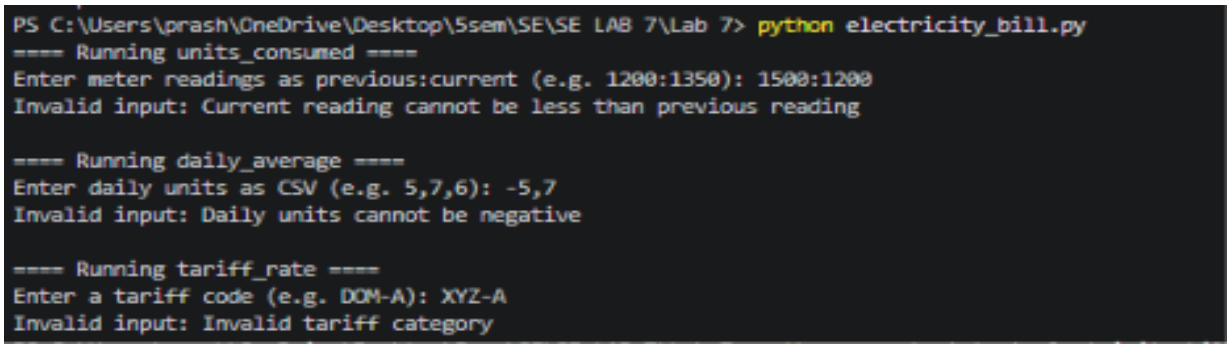
Step 8: Manual Invalid Input Testing

The application was also tested with invalid inputs.

Input	Result
1500:1200	Invalid meter reading
-5,7	Negative daily units rejected
XYZ-A	Invalid tariff category

The application displayed proper error messages without crashing.

Screenshot 5 – Invalid Input Handling



4. Result

The fuzz testing experiment was completed successfully. Three input validation bugs were identified and corrected. After fixing the application, all three test functions passed.

5. Conclusion

Through this lab, I learned how fuzz testing helps identify errors caused by unexpected inputs. I also understood how Hypothesis and Pytest can be used together to improve application reliability and input validation.